Assignment 2: Machine Learning & Patient Readmission Analysis

1. Overview & Assignment Purpose

When hospital patients with chronic conditions like diabetes are discharged, some end up returning to the hospital shortly afterward. Predicting which patients are at higher risk of returning allows healthcare teams to provide extra support and follow-up care before the patient leaves.

This aim is for this assignment is to build machine learning models to identify patients who might need additional attention after discharge against a hospitalization dataset. There is no single "correct" modeling recipe. You will explore the data, make thoughtful decisions about cleaning and splitting it, test different algorithms, and explain why your approach makes sense.

Note that the submission against the assignment is highly standardized for evaluation purposes. What's Standardized: the prediction target, the training data, and the submission format (Sections 2, 3 and 5). This will allow evaluation of your submitted model against a set of hidden patients. Your options regarding how to build your model and pre-process it are completely open.

2. The Prediction Task (Common to All Teams)

All teams must predict early readmission:

- Positive class (1): readmitted == "<30" (patient returned within 30 days of discharge).
- Negative class (0): readmitted is ">30" or "NO".

Your model must output, for each hospital stay (row), the predicted probability that the stay is followed by an early readmission. You are free to explore other framings (any readmission, three-class prediction) as an optional extension in your report, but your submitted model is evaluated only on the binary early-readmission task stated above.

3. Accessing the Dataset

Use the development file provided along with the assignment ***"diabetes_dev.csv"*** (with ***"IDS_mapping.csv"*** for ID descriptions). Note that the held-out data uses the same raw format, so your pipeline must handle raw values itself.

4. Open-Ended Directions to Explore

Decide as a team how you will address each area:

1. Defining Your Prediction Target: The target is fixed (Section 2). In your report, explain why early readmission is a practical framing for hospital staff, and discuss any limitations (e.g., class imbalance, what a positive prediction should trigger).
2. Cleaning & Grouping Features: Look closely at the raw values. Notice how missing information is recorded (such as "?") and how columns like diagnoses contain hundreds of specific codes. Explore ways to simplify, clean, or combine these features into broader categories. Remember that all cleaning must be reproducible on new raw data (Section 5).
3. Thoughtful Data Splitting & Validation: Some patients appear in multiple rows (patient_nbr). If the same patient appears in both training and validation data, your results will be unrealistically optimistic. Consider your options for sorting this out.
4. Exploring Machine Learning Algorithms: Do not rely on a single model. Compare simple baselines with more flexible algorithms, and discuss the trade-offs between simplicity, interpretability, and predictive performance.
5. Evaluating Errors & Decision Cutoffs: Failing to spot a high-risk patient is not the same as giving extra follow-up to someone who turns out fine. Experiment with decision thresholds beyond 50%, and justify the single threshold you submit (see Section 5). Choose it using your development data only.
6. Checking Performance Across Patient Groups: Investigate whether your model performs equally well across age groups and other demographic categories (e.g., gender, race). Report subgroup sample sizes alongside performance, and discuss which groups show higher error rates.

5. Submission Standard (Required)

Final models will be re-run by the instructor on a hidden, patient-disjoint held-out set. To make this possible, every team must follow the structure and interface below exactly. Submissions that cannot be run as specified may receive zero for the held-out evaluation component.

5.1 Submission package

Submit a single .zip file named group_<ID>.zip (e.g., group_07.zip) with exactly this layout:

Python Folder:

```
group_<ID>/
  notebook.ipynb        # full exploration, training, evaluation (runs top to bottom)
  report.pdf            # 4-5 page executive summary
  model/                # trained, fitted pipeline + any artifacts it needs
  predict.py            # Python file used for held-out dataset prediction
  threshold.json        # {"threshold": 0.xx}
  requirements.txt      # Python: pinned versions
  README.md             # 5-10 lines: language/version, how to run, team members
```

R Folder:

```
group_<ID>/
  predict.r             # required interface with fill in instructions. Must be .r file
  analysis.rmd          # analysis/training template
  model/                # trained, fitted pipeline.rds + any artifacts it needs
  threshold.json        # {"threshold": 0.xx}
  env.txt               # R: sessionInfo() output in
  README.md             # 5-10 lines: language/version, how to run, team members
```

5.2 Prediction interface

Your preprocessing and model must be bundled so that predictions can be made directly from raw, unlabeled rows, in the same format as diabetes_dev.csv but without the readmitted column. All cleaning, grouping, encoding, and imputation must happen inside your predict code or your saved pipeline. Do not rely on state left in your notebook or markdown file.

Python teams: predict.py must define the function below and support the command line shown.

```
# predict.py
import pandas as pd, numpy as np

def predict_proba(df: pd.DataFrame) -> np.ndarray:
    """df: raw rows (same columns as diabetes_dev.csv, no "readmitted").
    Returns a 1-D array, same length and order as df, of probabilities
    in [0, 1] that the stay is followed by readmission within 30 days."""
    ...

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--input", required=True)
    ap.add_argument("--output", required=True)
    a = ap.parse_args()
    df = pd.read_csv(a.input)
    out = pd.DataFrame({"encounter_id": df["encounter_id"],
                        "prob_early_readmit": predict_proba(df)})
    out.to_csv(a.output, index=False)
```

Run by the instructor as: python predict.py --input holdout_features.csv --output preds.csv

R teams: predict.R must define predict_proba(df) returning a numeric vector, and be runnable as shown.

```
# predict.R
predict_proba <- function(df) {
  # df: raw rows, no "readmitted" column. Return numeric vector, same length/order as
df
}
```

```
args <- commandArgs(trailingOnly = TRUE)   # input.csv output.csv
df <- read.csv(args[1], stringsAsFactors = FALSE)
out <- data.frame(encounter_id = df$encounter_id,
                  prob_early_readmit = predict_proba(df))
write.csv(out, args[2], row.names = FALSE)
```

Run by the instructor as: Rscript predict.R holdout_features.csv preds.csv

5.3 Rules your code must follow

- Load the model from the model/ folder using relative paths only (no absolute paths such as C:\Users\...). Save Python pipelines with joblib (a scikit-learn Pipeline, including preprocessing) and R models with saveRDS.
- Never retrain inside predict code. Prediction must be deterministic, and must not need internet access or the training data.
- Do not use encounter_id or patient_nbr as predictive features. Held-out patients are unseen, so these cannot help and may break your code.
- Handle unseen categories, "?" values, and missing columns' values gracefully. The held-out data may contain category levels (e.g., rare diagnosis codes) not present in the development data. Your code must not crash on them.
- Preserve row count and order: one probability per input row, in [0, 1], with no NaN values.
- Use only widely available libraries (Python: numpy, pandas, scikit-learn, xgboost, lightgbm, scipy, joblib; R: standard CRAN packages). Pin versions in requirements.txt (Python) or report sessionInfo() in env.txt (R). Record your language version in README.md.
- Runtime: predictions for ~20,000 rows must finish in under 5 minutes on an ordinary laptop (no GPU).

5.4 Threshold

Save the single decision threshold you recommend in threshold.json, for example {"threshold": 0.15}. It must be chosen using development data only, and justified in your report. The instructor will apply it to your predicted probabilities on the held-out set. Do not change it after submission.

5.5 Dry-run self-check (strongly recommended)

Before submitting, extract a 500-row sample of diabetes_dev.csv, drop the readmitted column, save it as sample_features.csv, and run your predict command from a fresh terminal/session in an empty folder containing only your unzipped package. If it fails there, it will fail when we run it.

6. Deliverables

1. Executable notebook (notebook.ipynb/ analysis.rmd): clean, well-commented code showing exploration, cleaning, model training, and evaluation. It must run top to bottom without errors, using diabetes_dev.csv.

2. Executive summary report (report.pdf): a concise 4-to-5 page report summarizing your design choices, validation strategy, model comparison, threshold justification, and patient subgroup findings. Include your own patient-grouped validation results.
3. Reproducible model package: model/, predict.py or predict.R, threshold.json, requirements.txt/env.txt, README.md as in Section 5.

7. Evaluation

The instructor will run your submitted predict script on the hidden held-out patients and compute the following (these are never shared as row-level feedback):

Metric	**Use**
Average precision (primary)	Ranking quality for the early-readmission class
ROC-AUC	Overall discrimination
Brier score	Probability calibration
Precision, recall, F1, confusion matrix at your submitted threshold	Practical performance of your chosen operating point
Metrics by age group / gender / race (with subgroup sizes)	Fairness check
Gap between your validation results and held-out results	Detects leakage or overfitting (e.g., non-grouped validation)

Oral Viva Defense (Scaling Factor Multiplier V):

Multiplier	**Understanding Level**	**Defense Expectations**
V = 1.00	Full Code Ownership	Team clearly explains their code, justifies how they split data and cleaned features, and understands why their models performed as they did.
V = 0.70 - 0.85	Partial Understanding	Team explains general concepts but struggles to explain specific coding choices, feature transformations, or validation behavior.
V = 0.00 - 0.50	Uncritical Adoption	Team cannot explain how their code works or fails to justify basic modeling and data cleaning decisions.

Note: Your final assignment grade will be calculated as: **Final Grade = Report Grade × Viva Multiplier (V)**. This ensures that every team member takes active ownership of the code and analysis.

8. Academic Integrity

- Use only the provided development file for model building. Using the full public dataset or attempting to infer held-out labels is prohibited.
- Evaluation attempts are limited to the final submission. No intermediate scoring or row-level feedback will be provided.
- Your code and results may be questioned in the viva. Unexplained results far above typical performance will be investigated.